MODULE

08

Stash, tags & releases

Switch context in seconds, name your releases, and answer ”what changed?”.

YOU WILL LEARN

git stash	01_stash.md
Tags & SemVer	02_tags_and_semver.md
Release notes	03_release_notes.md
Exercise: hotfix release	ex08_hotfix_release/
Interview Q&A	INTERVIEW.md

WHY STASH

Interrupted in the middle of your work

THE PROBLEM

You’re halfway through a feature when the boss says: **”Production is broken, fix it now!”** Your changes aren’t ready to commit, and Git won’t let you switch branches.

THE IDEA

git stash sweeps your unfinished work into a drawer. Fix the bug, come back, and git stash pop it out again.

TRY IT STASH

Put work aside, switch, come back

```
$ git status --short
 M cart.py
?? coupon.py
$ git switch hotfix/rounding
error: Your local changes to the following files would be overwritten by checkout:
        cart.py
Please commit your changes or stash them before you switch branches.
Aborting
$ git stash push -u -m "coupon WIP"
Saved working directory and index state On main: coupon WIP
$ git switch -q hotfix/rounding
$ git switch -q main
$ git stash pop -q
$ git status --short
 M cart.py
?? coupon.py
```

1. Git refuses to switch: your edit would be lost.
2. **-u** includes the new file coupon.py.
3. **pop** restores the work and removes the stash.

STEP BY STEP STASH

Stash commands

COMMAND	DOES
`git stash push -u -m "msg"`	save changes + new files, clean the tree
`git stash list`	stash@{0} (newest), stash@{1}, …
`git stash pop`	re-apply the newest and drop it
`git stash apply stash@{1}`	re-apply one and keep it
`git stash show -p stash@{0}`	see what’s inside
`git stash branch fix-later`	turn a stash into a branch

> Longer than a day? Commit on a branch instead: stashes are easy to forget.

PICTURE IT TAGS

A tag is a bookmark that never moves

🏷️ A **branch** moves forward with every commit. A **tag** stays on one commit forever: perfect for "this is exactly what we released as v1.2.0".

lightweight
git tag v1.0.0: just a name

→

annotated
git tag -a v1.0.0 -m ...: tagger, date, message

→

push
git push origin v1.0.0 (not automatic!)

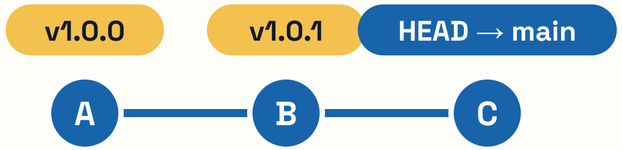
→

release
CI builds + GitHub Release

PICTURE IT TAGS

Tags mark releases

Tags vs branches



1. **v1.0.0** and **v1.0.1** stay where they are forever.
2. **main** moves with every new commit.
3. **git describe**: v1.0.1-1-g<id> = 1 commit after v1.0.1.

TRY IT TAGS

Annotated tags and git describe

```
$ git tag -a v1.0.0 -m "First public release"
$ git cat-file -t v1.0.0
tag
$ echo "wishlist" > wishlist.txt
$ git add wishlist.txt
$ git commit -qm "feat: wishlist"
$ git describe --tags
v1.0.0-1-g2d2ebce
$ git push origin --follow-tags
To ../github/shop.git
   b14578b..2d2ebce  main -> main
 * [new tag]         v1.0.0 -> v1.0.0
```

1. **cat-file -t**: an annotated tag is its own object.
2. **describe**: v1.0.0-1-g<id> = 1 commit after v1.0.0.
3. **--follow-tags** pushes annotated tags along with the commits.

STEP BY STEP SEMVER

Semantic Versioning: MAJOR.MINOR.PATCH

CHANGE	EXAMPLE	BUMP
`bug fix only`	fix: round totals	1.4.2 → **1.4.3**
`new feature, backwards compatible`	feat: wishlist	1.4.2 → **1.5.0**
`breaking change`	feat!: new API format	1.4.2 → **2.0.0**
`pre-release`	release candidate	**2.0.0-rc.1**

> Conventional Commits (Module 7) make the bump automatic.

TRY IT RELEASE NOTES

What's in v1.1.0?

```
$ git log --oneline v1.0.0..v1.1.0
782bd65 feat: wishlist
6b0a5bf fix: rounding in cart
5057df7 feat: coupons
$ git shortlog -sn v1.0.0..v1.1.0
     2  Priya
     1  Vishal
```

1. **A..B** = commits in B that aren't in A.
2. **shortlog -sn**: who contributed.
3. Paste into the GitHub Release notes.

COMMON MISTAKE TAGS

Moving a public tag

✗ WRONG

```
git tag -f v1.2.0 HEAD
git push --force origin v1.2.0
# customers' v1.2.0 != your v1.2.0
```

Anyone who already fetched v1.2.0 keeps the old commit. Two different ”v1.2.0” builds now exist: a support nightmare.

✓ RIGHT

```
# fix it with a NEW version
git tag -a v1.2.1 -m "Fix packaging"
git push origin v1.2.1
```

Released tags are immutable. Mistake? Ship the next patch version.

IN REAL JOBS RELEASES

Releases at work

Cut a release

```
git tag -a v2.3.0 -m "Release 2.3.0"
git push origin v2.3.0
```

CI on tag push

```
on: push: tags: ["v*"]
# build, test, publish, GitHub Release
```

Which version is deployed?

```
git describe --tags   # v2.3.0-4-g1a2b3c
```

Quick context switch

```
git stash -u && git switch hotfix
...
git switch - && git stash pop
```

INTERVIEW SAY THIS IN THE INTERVIEW

“Lightweight vs annotated tags? How do you release?”

- A **lightweight** tag is just a name; an **annotated** tag stores tagger, date and message, and can be signed.
- I use annotated tags with **SemVer**: v2.3.0.
- Tags aren’t pushed automatically: **git push origin v2.3.0** or **--follow-tags**.
- Pushing the tag triggers CI to build, publish and create the GitHub Release.
- Released tags never move; mistakes get a new patch version.

> **Follow-up they ask next:** ”What does git stash do?” (saves uncommitted changes on a stack and cleans the tree; -u for untracked; pop restores).

RECAP REMEMBER THESE

Module 8 in 5 lines

1. **git stash push -u** → switch → **git stash pop**.
2. Tags name releases; use **annotated** tags.
3. Push tags explicitly: **origin v1.2.0** / **--follow-tags**.
4. SemVer: **MAJOR.MINOR.PATCH**.
5. **log A..B** + **shortlog -sn** = release notes.

PRACTICE NOW

Do exercises/ex08_hotfix_release/: stash, tag, restore.